

Tampere University

Unit of Computing Sciences

COMP.SE.610 Software Engineering Project 1

COMP.SE.620 Software Engineering Project 2

G20

Snowledge

Final Report

An Nguyen
Chinh Pham
Khoa Nguyen
Phuong Nguyen
Pinja Valtanen
Tuomas Metsoila

Version history

Version	Date	Author	Description
0.1	27.11.2013	Chinh Pham	First draft, section 1, 2, 3
0.2	10.12.2023	Khoa Nguyen (PO)	Section 1 & 3
0.3	13.12.2023	Chinh Pham	Continue section 3, 6, 7
0.4	15.12.2023	Chinh Pham	Continue section 3, 4, 5, 8, 9, 10
1.0	15.12.2023	Chinh Pham	Finalize report
1.1	18.12.2023	Chinh Pham	Adding more detailed on deliverables

Contents

1	Introduction	4
1.1	Purpose of the report.....	4
1.2	Product and environment	4
1.3	Definitions, abbreviations, and acronyms	5
2	Project organisation	5
2.1	Group.....	5
2.2	Customer.....	5
2.3	Other stakeholders	6
3	Project implementation.....	6
3.1	Communication	6
3.2	Tools and technologies.....	6
3.3	Sprints.....	7
3.4	Deliverables and outcomes	8
3.5	Restrictions and limitations	9
3.6	Third party components, licenses, and IPRs.....	9
4	Working hours.....	9
4.1	Personal contributions to project	10
4.2	Peer feedback	11
5	Quality assurance	12
5.1	General description of testing	12
5.2	Bug reporting.....	12
5.3	Conclusions on product's quality	13
6	Risks and problems.....	14
6.1	Foreseen risks.....	14
6.2	Risks not foreseen.....	14
7	Not implemented in this project	14
7.1	Rejected ideas	14
7.2	Further development	15
8	Lessons learnt.....	15
9	Comments about the course	16
10	Statistics	16

1 Introduction

1.1 Purpose of the report

This document is the final report for the software engineering course' project in Autumn 2023. Our application in this project is Snowledge. This document will describe the outcomes of the project, in terms of what deliverables we produced and what improvements we made to the product. The report will also highlight how the project performs through our test-driven development process, as well as the results from our testing – including a usability test. Lastly, this document will also cover our learning results from our time working on this project together.

1.2 Product and environment

Our team worked together on the Snowledge application for this project. Snowledge is an outdoor application published by Pallaksen Pöllöt. The application is used primarily as a mobile app on mobile devices. As of now, it is only available on Android, and there are plans to publish the application to Apple's App Store. There is also a web client for the application, with a more limited functionality compared to the mobile version.

The basic purpose of the application is to offer a rescue element for users in an emergency so that they can call for help from other nearby users in the area. Premium users of the application can also view information on the current snow and weather conditions in the Pallas area. Typical application users are skiers or other explorers in the Pallas area who want to know about the environment and conditions in the area.

The rescue element is only presented on the mobile version of the product, since it uses the GPS location service of the device to aid the rescuing process. Using this feature, users of the mobile application can send a help request straight from their phone, and the app will notify nearby users about the incident. Users can specify their problem, ranging from equipment problems, loss, to injuries. This information is also available for nearby users, personnel of Pallaksen Pöllöt, and the National Rescue Service. The mobile application also provides a real-time messaging method for communication between the help requester and the rescuers.

1.3 Definitions, abbreviations, and acronyms

API	Application Programming Interface
MMT	Metrics monitoring tool
PM	Project Manager
PO	Product Owner
UI	User interface.
UX	User experience.

2 Project organisation

2.1 Group

Table 2.1. Group members list

Members	Roles	Contact	Technical skills	Weekly work hours
Chinh Pham	PM, Senior Developer	Chinh.pham@tuni.fi +358 408731508	Qt/C++, UI/UX	9 hours
An Nguyen	PO, Senior Developer	An.nguyen@tuni.fi	SQL, Python	9 hours
Khoa Nguyen	PO, Senior Developer	khoa.h.nguyen@tuni.fi	React, Angular, Cloud	9 hours
Tuomas Metsoila	Developer	tuomas.metsoila@tuni.fi	C#/.NET	9 hours
Pinja Valtanen	Designer, Developer	pinja.valtanen@tuni.fi	UI/UX, Python, JS	9 hours
Phuong Nguyen	Senior Developer	phuong.h.nguyen@tuni.fi	React, Angular, .NET	9 hours

2.2 Customer

Table 2.2. Customers' contact

Name	Email	Phone
Juuso Holstein	juuso@pallaksenpollot.com	040 5425544
Arto Hamalainen	artoahamalainen@gmail.com	

2.3 Other stakeholders

The project is organised as a university course and being supervised by course lecturers.

Coach Name: Pekka Mäkiahö

Email: pekka.makioho@istekki.fi

Emergency contact via Whatsapp: 050-5566266

3 Project implementation

3.1 Communication

1. Weekly meeting with customers:
Purpose: Updating and reviewing work progress with customers, collecting feedback from customers on project issues.
Schedule: Every Thursday at 4pm
Participants: Project team members and customers
Facilitator: Project manager
Tool: Microsoft Teams
2. Sprint review and planning:
Purpose: Inspect the outcome the sprint and determine future adaptation.
Planning for next sprint.
Schedule: bi-weekly on Thursday at 4pm
Participants: Project team members and coach
Facilitator: Project Manager
Tool: Microsoft Teams
3. Weekly internal team meetings:
Purpose: Project team internal discussions on project work progress.
Schedule: Every Tuesday at 4pm (may be changed to adapt to members' schedule)
Participants: Project team members
Facilitator: Project Manager
Tool: Microsoft Teams
4. Other communications
Team's internal communication: Telegram, Slack
Team – customers: Email and Slack
Team – course's personnel: Email
API technical assistants: Email

3.2 Tools and technologies

- Development tools: Flutter 3.7.0, Android studio, React.js, Flask (Python), MySQL database, and Docker. We used Flutter to implement the mobile front-end client of the application. Android Studio runs the mobile device emulators necessary for running the mobile version locally. React.js is used for the implementation of the web client of the application, and various web views are used

for the mobile app. The product uses a Python Flask back-end server and MySQL for its database. We also implemented a dockerized image to run the application locally within a Docker container without installing and setting up all the dependencies.

- Communication tools: Email for general communication purposes, Teams for conducting meetings with customers and coaches, Slack for communication between the team and the customers, and Telegram for private communication within team members.
- Version control system: GitLab. We had to use GitLab because the existing code based of the product is hosted on GitLab. It provides a CI/CD pipeline that we configured to automatically run our test suite whenever there are new changes introduced to the system.
- Process management: Jira, which includes the development Kanban board, features ticket system, tasks backlog, and project timeline visualization.
- Documentation: product documentation is stored on GitLab's b's repository wiki page. Course/project documentation is stored on Confluence, which can be accessed from Jira.
- Risk management: MMT for general risks. Sprint-specific risks are documented on Confluence.
- Weekly reporting: MMT
- Working hour tracking: MMT

3.3 Sprints

The project consists of 7 sprints in totals, which includes requirement gathering sprint (sprint 0), six development sprints (1-6). Testing has been performed during the entire 6 development sprints alongside implementation work.

During the project, there were 15 items in the product backlog with 53 tasks in all the sprints backlog and two rejected items. At the end of the project, 14 items were implemented, and 1 item that is in progress but not finished yet. The last item in the backlog is publishing the application to Apple Store but due to late delivery of needed hardware, the item must be left undone. However, the task was agreed with coach and clients to be personal task after the project for member that still lack work hour.

There is no data about how many tasks were not implemented during a certain sprint because team moved all tasks that were still in "Doing" when the sprint ended to the next sprint. Backlog prioritization worked well, and all the key features were implemented.

Here is a list that how many tasks were implemented during specific sprint:

- Sprint 0: 0 items
- Sprint 1: 1 item, 4 tasks
- Sprint 2: 2 items, 5 tasks
- Sprint 3: 2 items, 7 tasks
- Sprint 4: 3 items, 10 tasks
- Sprint 5: 4 items, 10 tasks
- Sprint 6: 3 items, 17 tasks

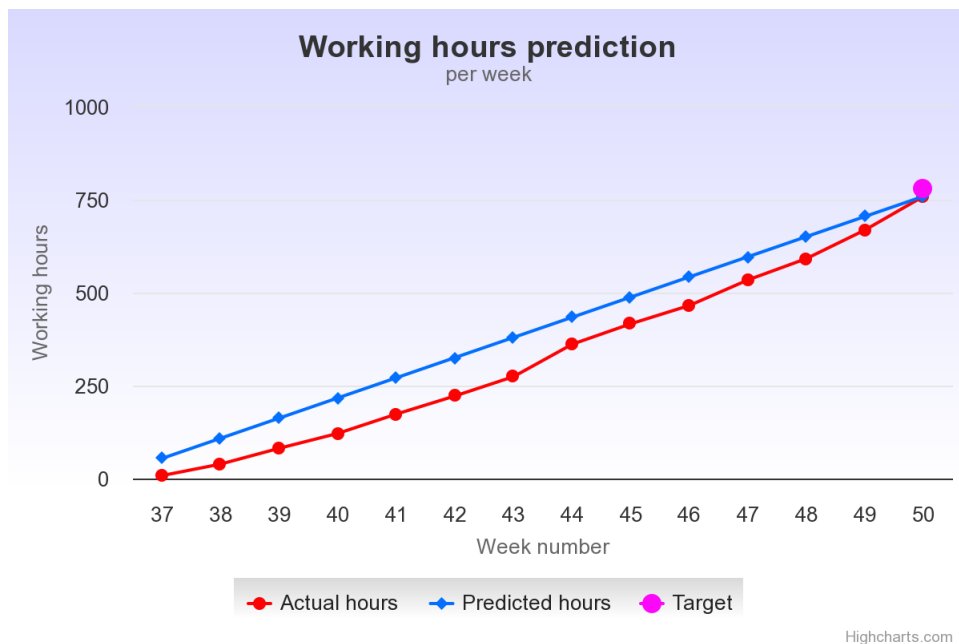


Image 3.1. Working hour prediction

3.4 Deliverables and outcomes

Features delivered in this project:

1. Mobile app divided into rescue and snow condition
2. Mobile app user guide interactive tutorial
3. New Snower API integration
4. Map compass added to current help view map
5. Automated chat message when the battery level is low
6. CI/CD implementation on Gitlab
7. Other general UX/UI improvements:
 - ✓ Adding problem type for helping request
 - ✓ Moving close button to the top of chat box
 - ✓ Changing help Decline/Accept button to a more accessible color
 - ✓ Hide chat bubble when help has not been accepted yet

The product is a finished application published to Google's Play Store after all the quality assurance is completed. Customers are responsible for publishing the app's package to the Play Store. The mobile package is available by building the project's source code using Flutter's default bundler, resulting in an .apk Android Package Kit file that can be submitted to the Play Store.

We also set up a new GitLab pipeline that is part of the CI process. This pipeline on GitLab will automatically run the defined Flutter and Python test suite, then attempt to build the Android version of the product whenever there are new changes introduced to the code base, e.g., new merge into master or new merge request into development/master branch. The test suite includes Flutter and Python's unit tests, and integration tests for the Python back-end server.

We conducted a usability test to evaluate the UX of our product and got some valuable insights from our volunteers users. Unfortunately, we did not able to contact the skiers in the Pallas, who are main users / potential users of the application.

Name	Statistics
Total lines of code	80594
Total lines of code (excluding JSON metadata, SVG and XML)	24796
New code lines	8001
Totally re-used	67.73% ($1 - 8001 / 24796$)
Partially re-used / New code lines	Aprox. 5%
File changes	66

3.5 Restrictions and limitations

There were some restrictions when we implemented the wanted functionalities of the product. These were mostly due to the poor architecture of the project, and the low overall cleanliness and quality of the existing system's code base.

3.6 Third party components, licenses, and IPRs

In this iteration, our team integrated the new API from data from Snower (app.snower.fi) to get the snow condition information from the new sensors they installed in the Pallas area. For this API, our customers contacted the administrators at Snower to provide us with the needed credentials and tokens to use it. Thus, the customers control the license and rights over the usage of this API in the product.

The application already has Maptiler (maptiler.com) services for all the maps inside the product. It is using the free version of this provider, and accessing this service is done through its provided API.

As for the mobile and web applications, the customers have all rights to them and their further development. Hosted on a private GitLab repository, the code base is not public, so the customers can decide what they want to do with the code and how they should manage it for the latter iteration of this project.

4 Working hours

Table 4.1. Realized working hours in person-hours by person and task.

	Chinh Pham	Tuomas Metsoila	Khoa Nguyen	Phuong Nguyen	Pinja Val-tanen	An Ngu-yen
Documen-tation	57.5	1.5	30.25	1	24	3.5
Require-ments	0	10.75	14.25	0	0	1.5
Design	2.5	0	0	0	26.5	0
Implemen-tation	1	52.25	25.75	66.25	13.5	80
Testing	6.5	13.5	21	9	8.25	21
Meetings	47.5	26.5	24.25	23.5	27.25	24.25
Studying	0	17	3	3.5	12	6.75
Other	12.5	2	14.5	0	10.5	1
Lectures	2	1.5	2.5	2	4	0
Total	129.5	125	135.5	105.25	153	138

Table 4.2. Realized working hours (weekly totals / project).

Week num-ber	Weekly total
Week 36	0
Week 37	8
Week 38	31
Week 39	43
Week 40	40
Week 41	52
Week 42	49
Week 43	52
Week 44	87
Week 45	55
Week 46	49
Week 47	69
Week 48	58
Week 49	77
Week 50	91
Total	761

4.1 Personal contributions to project

Chinh Pham, as our PM, managed every task related to management. He set up our Jira board issue trackers and Confluence to manage our team's documents. He also

planned the scope for each sprint, schedule Teams meeting, and communicate with our customer through emails. Chinh also did some design work for the front-end and writing documentation and preparing manual test cases.

Khoa Nguyen, our PO and senior developer, gathered requirements from customers and refined them to create new issues ticket for the product through Jira. He set up the CI pipeline for GitLab that enables automated unit and some integration test. He also coded on the Flutter code base for some features, designed and refactored to improve the existing code, whole also helped teammates whenever needed.

An Nguyen, our senior developer and PO, dealt with the back-end server and database of the product. He helped Khoa a bit when some requirements were related to the back-end or database. He fixed back-end bugs, ensured its server reliability, and defined more unit and integration test suite for the server and the database.

Pinja Valtanen, our designer, designed many prototypes for the new features requested by the customer. She also planned, conducted, and analyzed the results for the usability test. The test was really helpful for us, since it found many improvements we could make to enhance the overall UX of the product.

Phuong Nguyen, our senior developer, works on the Flutter front-end of the mobile application. She implemented the app-locking feature, the new tutorial feature, implemented the improvements from the usability test results, and fixed some mobile bugs.

Tuomas Metsoila, our junior developer, initially worked on the back-end server, but later on, worked mostly on the mobile Flutter front-end. He implemented some new features for the mobile version of the product, and help An with some of the back-end testing.

4.2 Peer feedback

The following is the summary of our team thought on peer feedback practices:

- Providing peer feedback was technically straightforward and uncomplicated.
- Receiving feedback at various project milestones is beneficial.
- In theory and potentially in application, feedback guides the project in the correct direction and enhances the performance of team members.
- When asked about areas for improvement, it is challenging to pinpoint specific skills, as individuals generally have awareness of their own strengths and weaknesses, which evolve as the project advances.
- Throughout the project, the team also offered verbal feedback to fellow group members, primarily acknowledging well-executed tasks.

5 Quality assurance

5.1 General description of testing

Our team follows a Test-Driven Development practice. When defining a feature ticket, we also define the required test to consider that feature completed. All newly implemented features must pass all test cases (both manual and available automation tests) before being merged into a common development branch of the project. At the end of the project, this branch will be going through system testing to ensure that all features are up and running and overall system reliability. All team members have responsibility of performing testing, specific testing tasks were assigned to members according to availability.

Usability testing was also conducted with 5 participants, who are mostly inexperienced users of the application, that help reveal many usability issues and inform improvement ideas.

Platform used for testings are android emulator and android mobile phone.

All test cases and test results were documented on Test Report.

5.2 Bug reporting

There was a total of 9 bugs identified during the project, 3 were introduced during the project implementation, and 6 were existing bugs in the system. 6 bugs were fixed and 3 are still left open. Below is the list of identified bugs with their status and severity. There are four severity levels from lowest to highest as follows: Low, Medium, High, Severe. Low-severity bugs are cosmetic issues or malfunctions that are minor and slightly affect user experience. Medium-severity bugs are malfunctions that severely affect user experience. High-severity bugs are malfunctions that prevent users from accessing an application service. Severe bugs are malfunctions that can shut down the system or prevent users from accessing multiple services. Bug can be rank higher if it is related to Rescue functionalities.

Table 5.1. Realized bugs list

ID	Description	Status	Severity
1	Cannot enter chat message in rescue mode chat room	Resolved	Severe
2	Mismatch selected side bar index on app init	Resolved	High
3	All app pages get repeatedly built when app init	Resolved	Severe
4	Help request confirmation screen not easy to see and overlap with chat box button	Resolved	Medium
5	Sometimes changing language from Suomi to English and vice versa is not possible	Resolved	Medium
6	Missing translation is causing application crash when starting help request	Resolved	Severe
7	Chat icon of helper still shows even when they left the room already	Open	Low
8	Helper's chat color not updated when they join the request	Open	Low

9	Find my location button and ruler being multiplied on idling	Open	Low
---	--	------	-----

5.3 Conclusions on product's quality

The state management of the applications is all over the place. Centralized state management would make the application much easier to understand and more bug resistant. The React web application could be combined with REDUX state container to force centralized state management. In Flutter, the appState should be used to store the state as it offers native support for centralized state management.

The use of UDP (User Datagram Protocol) with rescue backend communication seems to be unreliable for delivering help requests and other information especially in remote fell areas where internet connections are often weak. This makes it hard to avoid bugs that relate to the backend and frontend being out of sync with their state, because some messages might be lost without the sender application knowing anything about it. This screams for acknowledgement messages that would be used to confirm to the sender that their message has been delivered successfully. However, implementing those with UDP would not really make sense as then it would almost be like implementing TCP (Transmission Control Protocol) functionalities manually and the light-weight benefits of UDP would be mostly lost. Therefore, using TCP with a REST API would be the best solution, which has also been experimented by the Oulu team without success to make it work.

Information security might have problems, OWASP (Open Worldwide Application Security Project) security standards should be checked.

Using the web application through a WebView on mobile is not very maintainable architecturally. Sharing stateful information, like the selected language, is very hard between two separate applications, especially because every web window runs their own instance of the React application. Since windows have their own app instances, every time one of the windows is opened from mobile, the window function to update the language in the React app must be run to make sure that the newest language state is updated in the web view content. Running the window function of the React app's document must be done using the runJavaScript method in Flutter. Other architecture concerns would be, for example, using separate backends and databases for the web and mobile applications.

Bad variable naming makes code hard to understand. Especially in server communications where the messages are handled with non-describing variable names like `result[0]` etc.

6 Risks and problems

6.1 Foreseen risks

Foresee Risks
Technology unfamiliarity among team members Buggy code Inaccurate estimation of tasks Team members busy schedules Bad communication among project team members Low productivity of team members Members temporary or permanently missing from Hardware difficulties (lacking suitable hardware) Lack of stakeholders' engagement Scope Creep

6.2 Risks not foreseen

Unforesee Risks
Uneven distribution of work among team members Incompatibility due to old version of component technologies used in the product Chosen technology is not suitable for certain development purposes Unrealized limitation in existed architectural design

7 Not implemented in this project

7.1 Rejected ideas

Flutter's `showcaseview` library was proposed, but later proved to not be suitable for development purpose of implementing tutorial on both mobile application and react client. Therefore, bubble showcase is being used instead.

Implementation of payment system is rejected by the team due to low customer interest for the time being and short in development time.

7.2 Further development

1. The app could really use some system overhaul, from an architectural perspective. The current architecture of the system with combination of Flutter app and React client is showing difficulties for application expansion.
2. Design guidelines / design system must be created for consistency in future design of new features (UI features)
3. Coding guidelines and practices must be created for consistency in source code when there will be many different teams working on this product.
4. Translation should be applied to all parts of the application. At the reporting time, there are still many places that is only available in Finnish or English.
5. Thorough source code refactoring is needed as it will save a lot of time for future teams to take on the project
6. There are several UX/UI improvements ideas that we have drawn solutions to Figma that has not been implemented yet. (See "Known bug & UX/UI improvement" appendix)
7. Project dependencies should be upgraded to newer versions. Old versions of dependencies (Flutter, React, Python, etc.) may be outdated and create many incompatibilities to the application and time-consuming during development.

8 Lessons learnt

The team acquired knowledge of new technologies during the project, and everybody was given a chance to grow in their chosen field of expertise, such as Front-end and Back-end development, design, and management. Additionally, the team gained proficiency in version control and working in a Scrum team. The significance of effective meetings and communication within the team and with the customer became evident. The development of the mobile application and the enhancement of its visual appeal progressed successfully, receiving positive feedback from the customer.

The project appeared to be difficult to understand at first since team members were not familiar with the technologies used in the project and were not aware of what were already available and what was not in the application. When the project proceeded, everyone gradually learned more about the system and was able to propose improvement solutions. However, this unfamiliarity still caused us many problems, such as false estimation of workload, leading to false estimation of completion time and unbalanced distribution of work among team members. From this experience, we also realized that being flexible is important in project planning as there will always be raised problems that we have not thought of.

In this project, we realized that our communication within team and between team and clients were not always effective as misunderstandings happened quite often. Therefore, in a similar project next time, we will put more effort into communication and make sure that all stakeholders are on the same page.

9 Comments about the course

The course allowed us to have experience as close to real working environments as possible with real clients and meaningful project to work on. There are also dedicated coaches willing to help and guide us throughout the project implementation.

There is one downside in the course implementation is at the project selection stage. Our group was struggling with project selection as there was no way we could be sure about which topics were free to select and which were taken by another group. It is up to the group to upload a post saying that they have taken a topic, but some were not doing so. Because of this, we wasted time contacting clients from an already chosen topic, thus, starting the actual project work quite late.

10 Statistics

Group project: Snowledge

Member count: 6

Customer: Pallaksen Pollot

Tool lists: Flutter/Dart, ReactJS, Android Studio, Flask/Python, MySQL, Docker, Gitlab, Gitlab CI/CD, Jira, Confluence, Figma

Total lines of code: 24796

New lines of codes: 8001

Partially re-used / New code lines: Aprox. 5%

Number of new commits: 141

Number of Implemented items: 15 (1 undone)

Number of rejected items: 2

Number of tasks: 53

We have learned a lot from the project, especially in mobile and web development, working methods in Scrum and communication between stakeholders.

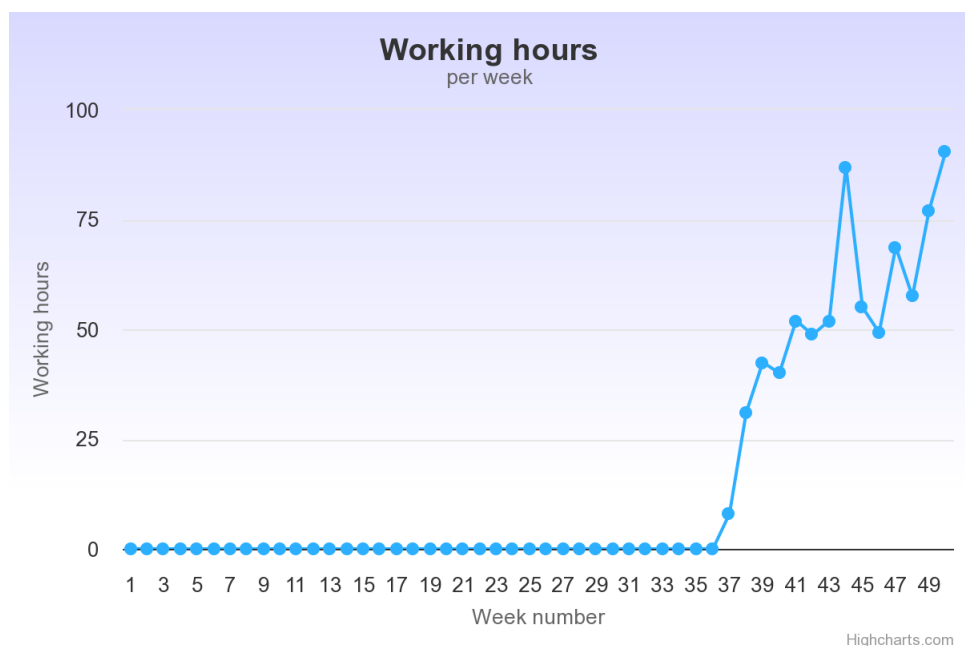


Image 10.1. Working hour per week

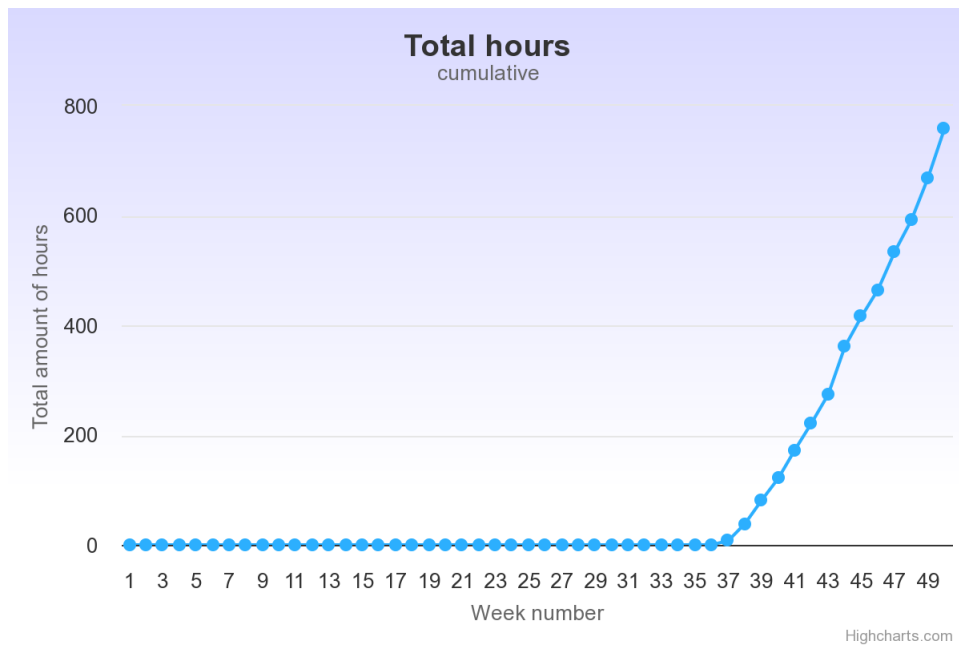


Image 10.2. Total working hour

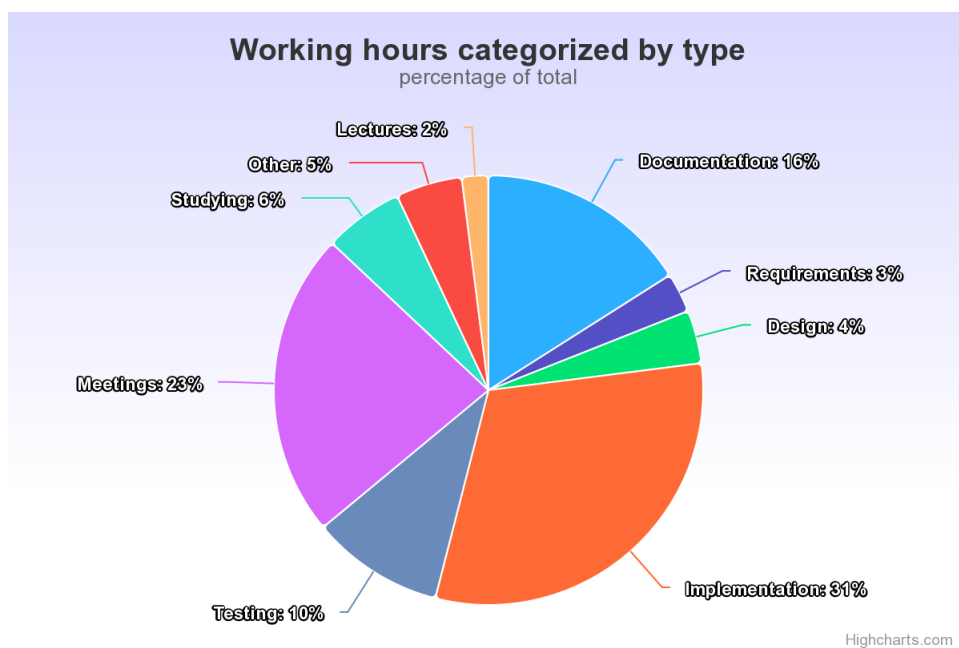


Image 10.3. Working hour categorized by type